

Denarii Dolor — Unit Test Scripts

This document lists the source of every JVM unit test script in `app/src/test/java/com/denariidolor/`: **23 files, 116 test methods**. Section 2 contains the scripts that implement the validation test plan (TC-01 – TC-20). Section 3 contains the rest of the unit suite, which runs alongside them as regression tests.

Each file's Apache-2.0 license header is omitted here for brevity; it is present in the repository.

1. Running the scripts

```
# validation scripts only
./gradlew testDebugUnitTest --tests "com.denariidolor.ValidateTransactionUseCaseTest" --tests
    "com.denariidolor.ValidatorsTest" --tests "com.denariidolor.util.MoneyTest"
# entire unit suite + coverage
./gradlew testDebugUnitTest createDebugUnitTestCoverageReport
```

In Android Studio, click the green run arrow beside a test class or method, or right-click `app/src/test` and choose **Run 'Tests in ...'**.

File	Tests	Purpose
<code>DateUtilsTest.kt</code>	3	Date formatting and month ranges
<code>GenerateReportUseCaseTest.kt</code>	3	Monthly report: title, timestamp, totals, rows, payment method
<code>LedgerTest.kt</code>	8	Polymorphic balance impacts of Expense/Income/Transfer and ledger deltas
<code>ManageReferenceDataUseCaseTest.kt</code>	8	Category/account/budget CRUD rules (duplicates, defaults, in-use)
<code>SearchFilterParserTest.kt</code>	7	Parsing of search inputs into SearchFilters
<code>SessionManagerTest.kt</code>	2	5-minute session timeout
<code>TransactionCrudUseCaseTest.kt</code>	5	Add/update/delete transaction use cases
<code>TransactionEntityMappingsTest.kt</code>	2	Entity ↔ domain model mapping
<code>TransactionModelTest.kt</code>	4	Transaction subclasses and TransactionType
<code>ValidateTransactionUseCaseTest.kt</code>	15	Transaction validation rules (TC-01 – TC-15)
<code>ValidatorsTest.kt</code>	3	Field validators: amount, description, names, percentages, search ranges (TC-16 – TC-18)
<code>data/local/db/DatabaseKeysTest.kt</code>	5	SQLCipher key format and handling

File	Tests	Purpose
data/local/preferences/SecurityProfileServiceTest.kt	17	PIN hashing, lockout escalation, recovery, wipe
domain/report/ReportFormattingTest.kt	4	Report labels, file names, CSV layout and formula-injection guard
presentation/ui/CategoryIconsTest.kt	3	Category icon catalogue
presentation/ui/DashboardMapperTest.kt	12	Dashboard summaries, chart data, budget progress
presentation/ui/FormMappingTest.kt	4	Form input ↔ entity mapping
presentation/ui/TransactionTypeDefaultsTest.kt	2	Form defaults per transaction type
presentation/viewmodel/DashboardViewModelTest.kt	1	Dashboard ViewModel state
presentation/viewmodel/TransactionViewModelTest.kt	4	Transaction ViewModel save/validation events
testutil/FakeRepositories.kt	0	In-memory test doubles and TestData fixture
testutil/MainDispatcherRule.kt	0	JUnit rule that swaps Dispatchers.Main for ViewModel tests
util/MoneyTest.kt	4	Money parsing/formatting in cents (TC-19 – TC-20)
Total	116	

2. Validation test scripts (test plan TC-01 – TC-20)

ValidateTransactionUseCaseTest.kt

Transaction validation rules (TC-01 – TC-15).

```
package com.denariidolor

import com.denariidolor.data.local.db.entity.BudgetEntity
import com.denariidolor.data.local.db.entity.CategoryEntity
import com.denariidolor.data.local.db.entity.TransactionEntity
import com.denariidolor.data.repository.CategoryRepository
import com.denariidolor.domain.model.TransactionType
import com.denariidolor.domain.usecase.ValidateTransactionUseCase
import com.denariidolor.testutil.FakeAccountRepository
import com.denariidolor.testutil.FakeBudgetRepository
import com.denariidolor.testutil.FakeCategoryRepository
import com.denariidolor.testutil.FakeTransactionRepository
import com.denariidolor.testutil.TestData
import kotlinx.coroutines.runBlocking
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertTrue
```

```

import org.junit.Test

class ValidateTransactionUseCaseTest {
    private val transactions = FakeTransactionRepository()
    private val budgets = FakeBudgetRepository(listOf(BudgetEntity(id = 1, categoryId = 4,
monthlyLimitCents = 10_000)))
    private val useCase = ValidateTransactionUseCase(
        FakeCategoryRepository(TestData.categories),
        FakeAccountRepository(TestData.accounts),
        budgets,
        transactions
    )

    @Test
    fun expenseFailsWhenBudgetExceeded() = runBlocking<Unit> {
        transactions.expenseTotalCents = 9_000

        val result = useCase(TestData.expense(amountCents = 2_000))

        assertTrue(result.isFailure)
        assertEquals("Budget threshold violated", result.exceptionOrNull()?.message)
    }

    @Test
    fun editedExpenseExcludesItsOwnPreviousAmountFromBudget() = runBlocking<Unit> {
        transactions.expenseTotalCents = 8_000

        val result = useCase(TestData.expense(id = 7, amountCents = 2_000))

        assertTrue(result.isSuccess)
        assertEquals(7L, transactions.lastExcludedTransactionId)
    }

    @Test
    fun transferFailsWhenDestinationMatchesSource() = runBlocking<Unit> {
        val result = useCase(
            TestData.expense().copy(type = TransactionType.TRANSFER, categoryId = 3, accountId = 1,
transferAccountId = 1)
        )

        assertTrue(result.isFailure)
    }

    @Test
    fun transferFailsWhenDestinationAccountMissing() = runBlocking<Unit> {
        val result = useCase(
            TestData.expense().copy(type = TransactionType.TRANSFER, categoryId = 3, accountId = 1,
transferAccountId = 99)
        )

        assertEquals("Transfer destination account not found", result.exceptionOrNull()?.message)
    }

    @Test
    fun failsWhenCategoryOrAccountMissing() = runBlocking<Unit> {
        assertEquals("Category not found", useCase(TestData.expense(categoryId =
99)).exceptionOrNull()?.message)
    }
}

```

```

        assertEquals("Account not found", useCase(TestData.expense(accountId =
99)).exceptionOrNull()?.message)
    }

    @Test
    fun expenseExactlyAtLimitIsAllowed() = runBlocking<Unit> {
        transactions.expenseTotalCents = 8_000

        assertTrue(useCase(TestData.expense(amountCents = 2_000)).isSuccess)
        assertTrue(useCase(TestData.expense(amountCents = 2_001)).isFailure)
    }

    @Test
    fun rejectsZeroOrNegativeAmount() = runBlocking<Unit> {
        assertEquals("Invalid amount", messageFor(TestData.expense(amountCents = 0)))
        assertEquals("Invalid amount", messageFor(TestData.expense(amountCents = -1)))
    }

    @Test
    fun rejectsBlankDescription() = runBlocking<Unit> {
        assertEquals("Description cannot be blank", messageFor(TestData.expense().copy(description
= " ")))
    }

    @Test
    fun rejectsNonPositiveDate() = runBlocking<Unit> {
        assertEquals("Invalid date", messageFor(TestData.expense().copy(dateEpochMillis = 0)))
    }

    @Test
    fun rejectsNonPositiveAccountOrCategoryIds() = runBlocking<Unit> {
        assertEquals("Invalid account", messageFor(TestData.expense(accountId = 0)))
        assertEquals("Invalid category", messageFor(TestData.expense(categoryId = 0)))
    }

    @Test
    fun transferRequiresDestination() = runBlocking<Unit> {
        val transfer = TestData.expense().copy(type = TransactionType.TRANSFER, categoryId = 3,
transferAccountId = null)

        assertEquals("Transfer destination is required", messageFor(transfer))
    }

    @Test
    fun validTransferPasses() = runBlocking<Unit> {
        val transfer = TestData.expense().copy(type = TransactionType.TRANSFER, categoryId = 3,
accountId = 1, transferAccountId = 2)

        assertTrue(useCase(transfer).isSuccess)
    }

    @Test
    fun budgetOnlyLimitsExpenses() = runBlocking<Unit> {
        transactions.expenseTotalCents = 9_000
        val income = TestData.expense(amountCents = 5_000).copy(type = TransactionType.INCOME)

        assertTrue(useCase(income).isSuccess)
    }

```

```

    }

    @Test
    fun expenseWithoutBudgetIsNotLimited() = runBlocking<Unit> {
        transactions.expenseTotalCents = 1_000_000

        assertTrue(useCase(TestData.expense(amountCents = 500_000, categoryId = 1)).isSuccess)
    }

    @Test
    fun repositoryErrorBecomesFailureInsteadOfCrash() = runBlocking<Unit> {
        val failingCategories = object : CategoryRepository by
        FakeCategoryRepository(TestData.categories) {
            override suspend fun getById(id: Long): CategoryEntity? = throw
            IllegalStateException("database closed")
        }
        val failingUseCase = ValidateTransactionUseCase(failingCategories,
        FakeAccountRepository(TestData.accounts), budgets, transactions)

        val result = failingUseCase(TestData.expense())

        assertFalse(result.isSuccess)
        assertEquals("database closed", result.exceptionOrNull()?.message)
    }

    private suspend fun messageFor(transaction: TransactionEntity): String? =
    useCase(transaction).exceptionOrNull()?.message
}

```

ValidatorsTest.kt

Field validators: amount, description, names, percentages, search ranges (TC-16 - TC-18).

```

package com.denariidolor

import com.denariidolor.domain.model.SearchFilters
import com.denariidolor.util.Validators
import org.junit.Assert.assertFalse
import org.junit.Assert.assertTrue
import org.junit.Test

class ValidatorsTest {
    @Test
    fun amountAndDescriptionValidation() {
        assertTrue(Validators.isValidAmount(1_000L))
        assertFalse(Validators.isValidAmount(0L))
        assertFalse(Validators.isValidAmount(-1L))
        assertTrue(Validators.isValidDescription("Rent"))
        assertFalse(Validators.isValidDescription(" "))
    }

    @Test
    fun rangeValidationRejectsInvertedRanges() {
        assertFalse(Validators.isValidSearchRange(SearchFilters(minAmountCents = 10_000,
        maxAmountCents = 5_000)))
    }
}

```

```

        assertFalse(Validators.isValidSearchRange(SearchFilters(startDateEpochMillis = 2L,
endDateEpochMillis = 1L)))
        assertTrue(Validators.isValidSearchRange(SearchFilters(minAmountCents = 100, maxAmountCents
= 200)))
    }

    @Test
    fun nameAndPercentValidation() {
        assertTrue(Validators.isValidName("Groceries"))
        assertFalse(Validators.isValidName(" "))
        assertFalse(Validators.isValidName("x".repeat(Validators.MAX_NAME_LENGTH + 1)))
        assertTrue(Validators.isValidPercent(80))
        assertFalse(Validators.isValidPercent(0))
        assertFalse(Validators.isValidPercent(101))
    }
}

```

util/MoneyTest.kt

Money parsing/formatting in cents (TC-19 – TC-20).

```

package com.denariidolor.util

import org.junit.Assert.assertEquals
import org.junit.Assert.assertNull
import org.junit.Test

class MoneyTest {
    @Test
    fun parsesCommonInputsToCents() {
        assertEquals(1_200L, Money.parseToCents("12"))
        assertEquals(1_250L, Money.parseToCents("12.5"))
        assertEquals(1_205L, Money.parseToCents(" 12.05 "))
        assertEquals(123_456L, Money.parseToCents("$1,234.56"))
        assertEquals(-2_550L, Money.parseToCents("-25.50"))
        assertEquals(1_200L, Money.parseToCents("12.000"))
    }

    @Test
    fun rejectsInvalidInputs() {
        assertNull(Money.parseToCents(""))
        assertNull(Money.parseToCents("abc"))
        assertNull(Money.parseToCents("1.005"))
        assertNull(Money.parseToCents("1e3"))
        assertNull(Money.parseToCents("9999999999"))
    }

    @Test
    fun formatsCents() {
        assertEquals("12.50", Money.toPlain(1_250))
        assertEquals("4.5", Money.toInput(450))
        assertEquals("250", Money.toInput(25_000))
        assertEquals("$0.05", formatMoney(5))
        assertEquals("-$1.00", formatMoney(-100))
    }
}

```

```

@Test
fun centsSumExactly() {
    val tenCents = Money.parseToCents("0.10")!!
    assertEquals(Money.parseToCents("0.30"), tenCents * 3)
}
}

```

testutil/FakeRepositories.kt

In-memory test doubles and TestData fixture.

```

package com.denariidolor.testutil

import com.denariidolor.data.local.db.entity.AccountEntity
import com.denariidolor.data.local.db.entity.BudgetEntity
import com.denariidolor.data.local.db.entity.CategoryEntity
import com.denariidolor.data.local.db.entity.TransactionEntity
import com.denariidolor.data.repository.AccountRepository
import com.denariidolor.data.repository.BudgetRepository
import com.denariidolor.data.repository.CategoryRepository
import com.denariidolor.data.repository.TransactionRepository
import com.denariidolor.domain.model.SearchFilters
import com.denariidolor.domain.model.TransactionType
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flowOf

class FakeTransactionRepository(initial: List<TransactionEntity> = emptyList()) :
    TransactionRepository {
    val items = initial.toMutableList()
    var expenseTotalCents = 0L
    var lastExcludedTransactionId: Long? = null
    private var nextId = (initial.maxOfOrNull { it.id } ?: 0L) + 1

    override suspend fun add(transaction: TransactionEntity): Long {
        val id = nextId++
        items += transaction.copy(id = id)
        return id
    }

    override suspend fun update(transaction: TransactionEntity): Boolean {
        val index = items.indexOfFirst { it.id == transaction.id }
        if (index < 0) return false
        items[index] = transaction
        return true
    }

    override suspend fun delete(id: Long): Boolean = items.removeAll { it.id == id }
    override suspend fun getById(id: Long): TransactionEntity? = items.firstOrNull { it.id == id }
    override fun getAll(): Flow<List<TransactionEntity>> = flowOf(items.toList())
    override fun search(filters: SearchFilters): Flow<List<TransactionEntity>> =
        flowOf(items.toList())
    override suspend fun getDateRange(startInclusive: Long, endInclusive: Long):
        List<TransactionEntity> =
        items.filter { it.dateEpochMillis in startInclusive..endInclusive }
}

```

```

    override suspend fun getExpenseTotalForCategory(
        categoryId: Long,
        startInclusive: Long,
        endInclusive: Long,
        excludeTransactionId: Long
    ): Long {
        lastExcludedTransactionId = excludeTransactionId
        return expenseTotalCents
    }

    override suspend fun countByCategory(categoryId: Long): Int = items.count { it.categoryId ==
categoryId }
    override suspend fun countByAccount(accountId: Long): Int =
        items.count { it.accountId == accountId || it.transferAccountId == accountId }
}

class FakeCategoryRepository(initial: List<CategoryEntity> = emptyList()) : CategoryRepository {
    val items = initial.toMutableList()
    private var nextId = (initial.maxOfOrNull { it.id } ?: 0L) + 1

    override suspend fun add(category: CategoryEntity): Long {
        val id = nextId++
        items += category.copy(id = id)
        return id
    }

    override suspend fun update(category: CategoryEntity): Boolean {
        val index = items.indexOfFirst { it.id == category.id }
        if (index < 0) return false
        items[index] = category
        return true
    }

    override suspend fun delete(id: Long): Boolean = items.removeAll { it.id == id }
    override suspend fun getById(id: Long): CategoryEntity? = items.firstOrNull { it.id == id }
    override fun getAll(): Flow<List<CategoryEntity>> = flowOf(items.toList())
    override suspend fun isDuplicateName(name: String, excludeId: Long): Boolean =
        items.any { it.id != excludeId && it.name.equals(name, ignoreCase = true) }
}

class FakeAccountRepository(initial: List<AccountEntity> = emptyList()) : AccountRepository {
    val items = initial.toMutableList()
    private var nextId = (initial.maxOfOrNull { it.id } ?: 0L) + 1

    override suspend fun add(account: AccountEntity): Long {
        val id = nextId++
        items += account.copy(id = id)
        return id
    }

    override suspend fun rename(id: Long, name: String): Boolean {
        val index = items.indexOfFirst { it.id == id }
        if (index < 0) return false
        items[index] = items[index].copy(name = name)
        return true
    }
}

```

```

    override suspend fun delete(id: Long): Boolean = items.removeAll { it.id == id }
    override suspend fun getById(id: Long): AccountEntity? = items.firstOrNull { it.id == id }
    override fun getAll(): Flow<List<AccountEntity>> = flowOf(items.toList())
    override suspend fun isDuplicateName(name: String, excludeId: Long): Boolean =
        items.any { it.id != excludeId && it.name.equals(name, ignoreCase = true) }
}

class FakeBudgetRepository(initial: List<BudgetEntity> = emptyList()) : BudgetRepository {
    val items = initial.toMutableList()
    private var nextId = (initial.maxOfOrNull { it.id } ?: 0L) + 1

    override suspend fun upsert(budget: BudgetEntity): Long {
        items.removeAll { it.categoryId == budget.categoryId }
        val id = if (budget.id > 0L) budget.id else nextId++
        items += budget.copy(id = id)
        return id
    }

    override fun getAll(): Flow<List<BudgetEntity>> = flowOf(items.toList())
    override suspend fun getByCategoryId(categoryId: Long): BudgetEntity? = items.firstOrNull {
        it.categoryId == categoryId }
    override suspend fun deleteByCategoryId(categoryId: Long): Boolean = items.removeAll {
        it.categoryId == categoryId }
}

object TestData {
    val categories = listOf(
        CategoryEntity(id = 1, name = "General Expense"),
        CategoryEntity(id = 2, name = "General Income"),
        CategoryEntity(id = 3, name = "Transfer"),
        CategoryEntity(id = 4, name = "Groceries")
    )
    val accounts = listOf(
        AccountEntity(id = 1, name = "Cash", balanceCents = 0L),
        AccountEntity(id = 2, name = "Savings", balanceCents = 0L),
        AccountEntity(id = 3, name = "Checking", balanceCents = 0L)
    )

    fun expense(id: Long = 0, amountCents: Long = 2_000, categoryId: Long = 4, accountId: Long = 1)
    = TransactionEntity(
        id = id,
        type = TransactionType.EXPENSE,
        description = "Coffee",
        amountCents = amountCents,
        categoryId = categoryId,
        accountId = accountId,
        dateEpochMillis = System.currentTimeMillis()
    )
}

```

3. Regression unit test scripts

DateUtilsTest.kt

Date formatting and month ranges (3 tests).

```
package com.denariidolor

import com.denariidolor.util.DateUtils
import java.time.ZoneId
import org.junit.Assert.assertEquals
import org.junit.Test

class DateUtilsTest {
    private val zoneId = ZoneId.of("UTC")

    @Test
    fun parseIsoDateToStartOfDayEpochMillisUsesInclusiveStart() {
        assertEquals(1_725_148_800_000L, DateUtils.parseIsoDateToStartOfDayEpochMillis("2024-09-01", zoneId))
    }

    @Test
    fun parseIsoDateToEndOfDayEpochMillisUsesInclusiveEnd() {
        assertEquals(1_725_235_199_999L, DateUtils.parseIsoDateToEndOfDayEpochMillis("2024-09-01", zoneId))
    }

    @Test
    fun formatLocalDateUsesZone() {
        assertEquals("2024-09-01", DateUtils.formatLocalDate(1_725_148_800_000L, zoneId))
        assertEquals("2024-08-31", DateUtils.formatLocalDate(1_725_148_800_000L, ZoneId.of("America/Phoenix")))
    }
}
```

GenerateReportUseCaseTest.kt

Monthly report: title, timestamp, totals, rows, payment method (3 tests).

```
package com.denariidolor

import com.denariidolor.data.local.db.entity.TransactionEntity
import com.denariidolor.domain.model.TransactionType
import com.denariidolor.domain.report.ReportText
import com.denariidolor.domain.usecase.GenerateReportUseCase
import com.denariidolor.testutil.FakeAccountRepository
import com.denariidolor.testutil.FakeCategoryRepository
import com.denariidolor.testutil.FakeTransactionRepository
import com.denariidolor.testutil.TestData
import com.denariidolor.util.DateUtils
import java.time.Clock
import java.time.Instant
```

```

import java.time.YearMonth
import java.time.ZoneId
import kotlin.coroutines.runBlocking
import org.junit.Assert.assertEquals
import org.junit.Test

class GenerateReportUseCaseTest {
    private val zone = ZoneId.of("UTC")
    private val clock = Clock.fixed(Instant.parse("2026-09-20T12:00:00Z"), zone)
    private val september = YearMonth.of(2026, 9)
    private val start = DateUtils.monthRangeEpochMillis(2026, 9, zone).first

    private val transactions = FakeTransactionRepository(
        listOf(
            TransactionEntity(
                id = 1,
                type = TransactionType.EXPENSE,
                description = "Rent",
                amountCents = 90_000,
                categoryId = 4,
                accountId = 1,
                dateEpochMillis =
                    start + 2_000
            ),
            TransactionEntity(
                id = 2,
                type = TransactionType.INCOME,
                description = "Salary",
                amountCents = 200_000,
                categoryId = 2,
                accountId = 1,
                dateEpochMillis =
                    start + 1_000
            ),
            TransactionEntity(
                id = 3,
                type = TransactionType.TRANSFER,
                description = "Savings",
                amountCents = 30_000,
                categoryId = 3,
                accountId = 1,
                transferAccountId = 2,
                dateEpochMillis =
                    start + 3_000
            ),
            TransactionEntity(
                id = 4,
                type = TransactionType.EXPENSE,
                description = "Old",
                amountCents = 5_000,
                categoryId = 99,
                accountId = 1,
                dateEpochMillis =
                    start - 1
            )
        )
    )
}

```

```

    )
    private val useCase = GenerateReportUseCase(
        transactions,
        FakeCategoryRepository(TestData.categories),
        FakeAccountRepository(TestData.accounts),
        clock
    )

    @Test
    fun reportHasTitlePeriodTimestampAndTotals() = runBlocking<Unit> {
        val report = useCase(september)

        assertEquals(ReportText.TITLE, report.title)
        assertEquals(september, report.period)
        assertEquals(clock.millis(), report.generatedAtEpochMillis)
        assertEquals(200_000L, report.totalIncomeCents)
        assertEquals(90_000L, report.totalExpenseCents)
        assertEquals(110_000L, report.netCents)
    }

    @Test
    fun rowsAreChronologicalWithNamesAndPaymentMethod() = runBlocking<Unit> {
        val rows = useCase(september).rows

        assertEquals(listOf(2L, 1L, 3L), rows.map { it.transactionId })
        assertEquals("Groceries", rows[1].categoryName)
        assertEquals("Cash", rows[1].paymentMethod)
        assertEquals("Cash → Savings", rows[2].paymentMethod)
    }

    @Test
    fun unknownCategoryFallsBackToId() = runBlocking<Unit> {
        val august = useCase(YearMonth.of(2026, 8)).rows.single()

        assertEquals("#99", august.categoryName)
    }
}

```

LedgerTest.kt

Polymorphic balance impacts of Expense/Income/Transfer and ledger deltas (8 tests).

```

package com.denariidolor

import com.denariidolor.domain.model.Expense
import com.denariidolor.domain.model.Income
import com.denariidolor.domain.model.Ledger
import com.denariidolor.domain.model.Transfer
import org.junit.Assert.assertEquals
import org.junit.Test

class LedgerTest {
    private val expense = Expense(id = 1, description = "Food", amountCents = 4_500, categoryId = 1, accountId = 1, dateEpochMillis = 1L)

```

```

@Test
fun addAppliesImpact() {
    assertEquals(mapOf(1L to -4_500L), Ledger.balanceDeltas(previous = null, current = expense))
}

@Test
fun deleteReversesImpact() {
    assertEquals(mapOf(1L to 4_500L), Ledger.balanceDeltas(previous = expense, current = null))
}

@Test
fun editAmountAppliesOnlyTheDifference() {
    assertEquals(mapOf(1L to -500L), Ledger.balanceDeltas(expense, expense.copy(amountCents = 5_000)))
}

@Test
fun editAccountMovesImpactBetweenAccounts() {
    assertEquals(mapOf(1L to 4_500L, 2L to -4_500L), Ledger.balanceDeltas(expense, expense.copy(accountId = 2)))
}

@Test
fun unchangedEditProducesNoDeltas() {
    assertEquals(emptyMap<Long, Long>(), Ledger.balanceDeltas(expense, expense.copy(description = "Lunch")))
}

@Test
fun changingTypeFromExpenseToIncomeDoublesSwing() {
    val income = Income(id = 1, description = "Refund", amountCents = 4_500, categoryId = 2, accountId = 1, dateEpochMillis = 1L)
    assertEquals(mapOf(1L to 9_000L), Ledger.balanceDeltas(expense, income))
}

@Test
fun transferMovesMoneyBetweenAccounts() {
    val transfer = Transfer(description = "Save", amountCents = 10_000, categoryId = 3, accountId = 1, transferAccountId = 2, dateEpochMillis = 1L)
    assertEquals(mapOf(1L to -10_000L, 2L to 10_000L), Ledger.balanceDeltas(null, transfer))
    assertEquals(mapOf(1L to 10_000L, 2L to -10_000L), Ledger.balanceDeltas(transfer, null))
}

@Test
fun centsAvoidFloatingPointDrift() {
    val tenCents = expense.copy(amountCents = 10)
    val total = (1..3).fold(0L) { acc, _ -> acc + Ledger.balanceDeltas(null, tenCents).getValue(1L) }
    assertEquals(-30L, total)
}
}

```

ManageReferenceDataUseCaseTest.kt

Category/account/budget CRUD rules (duplicates, defaults, in-use) (8 tests).

```
package com.denariidolor

import com.denariidolor.domain.usecase.AccountUseCases
import com.denariidolor.domain.usecase.BudgetUseCases
import com.denariidolor.domain.usecase.CategoryUseCases
import com.denariidolor.testutil.FakeAccountRepository
import com.denariidolor.testutil.FakeBudgetRepository
import com.denariidolor.testutil.FakeCategoryRepository
import com.denariidolor.testutil.FakeTransactionRepository
import com.denariidolor.testutil.TestData
import kotlinx.coroutines.runBlocking
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

class ManageReferenceDataUseCaseTest {
    private val transactions = FakeTransactionRepository(listOf(TestData.expense(id = 1, categoryId = 4, accountId = 3)))
    private val categoryRepo = FakeCategoryRepository(TestData.categories)
    private val accountRepo = FakeAccountRepository(TestData.accounts)
    private val budgetRepo = FakeBudgetRepository()
    private val categories = CategoryUseCases(categoryRepo, transactions)
    private val accounts = AccountUseCases(accountRepo, transactions)
    private val budgets = BudgetUseCases(budgetRepo, categoryRepo)

    @Test
    fun categoryAddTrimsAndRejectsDuplicatesCaseInsensitively() = runBlocking<Unit> {
        assertTrue(categories.add(" Dining ").isSuccess)
        assertEquals("Dining", categoryRepo.items.last().name)
        assertTrue(categories.add("groceries").isFailure)
        assertTrue(categories.add(" ").isFailure)
    }

    @Test
    fun categoryRenameAllowsKeepingOwnName() = runBlocking<Unit> {
        assertTrue(categories.update(4, "GROCERIES").isSuccess)
        assertTrue(categories.update(4, "Transfer").isFailure)
        assertTrue(categories.update(99, "Other").exceptionOrNull() is NoSuchElementException)
    }

    @Test
    fun categoryRenamePreservesIcon() = runBlocking<Unit> {
        val id = categories.add("Pets", "ic_pets").getOrThrow()
        categories.update(id, "Pet Care")

        assertEquals("ic_pets", categoryRepo.items.single { it.id == id }.iconName)
    }

    @Test
    fun categoryDeleteBlockedWhenDefaultOrInUse() = runBlocking<Unit> {
        assertEquals("Default categories cannot be deleted",
            categories.delete(1).exceptionOrNull()?.message)
    }
}
```

```

        assertEquals("Category is used by 1 transaction(s)",
categories.delete(4).exceptionOrNull()?.message)
        val id = categories.add("Temp").getOrThrow()
        assertTrue(categories.delete(id).isSuccess)
    }

    @Test
    fun accountAddValidatesNameAndBalance() = runBlocking<Unit> {
        assertTrue(accounts.add("Credit Card", -25_000).isSuccess)
        assertEquals(-25_000L, accountRepo.items.last().balanceCents)
        assertTrue(accounts.add("cash").isFailure)
    }

    @Test
    fun accountDeleteBlockedWhenDefaultOrInUse() = runBlocking<Unit> {
        assertTrue(accounts.delete(1).isFailure)
        assertEquals("Account is used by 1 transaction(s)",
accounts.delete(3).exceptionOrNull()?.message)
        val id = accounts.add("Temp").getOrThrow()
        assertTrue(accounts.delete(id).isSuccess)
    }

    @Test
    fun budgetSetValidatesAndReplacesExisting() = runBlocking<Unit> {
        val firstId = budgets.set(4, 20_000).getOrThrow()
        val secondId = budgets.set(4, 30_000, 90).getOrThrow()

        assertEquals(firstId, secondId)
        assertEquals(30_000L, budgetRepo.items.single().monthlyLimitCents)
        assertEquals(90, budgetRepo.items.single().warningThresholdPercent)
        assertTrue(budgets.set(4, 0).isFailure)
        assertTrue(budgets.set(4, 10_000, 0).isFailure)
        assertTrue(budgets.set(99, 10_000).isFailure)
    }

    @Test
    fun budgetDelete() = runBlocking<Unit> {
        budgets.set(4, 20_000)
        assertTrue(budgets.delete(4).isSuccess)
        assertTrue(budgets.delete(4).isFailure)
    }
}

```

SearchFilterParserTest.kt

Parsing of search inputs into SearchFilters (7 tests).

```

package com.denariidolor

import com.denariidolor.presentation.ui.search.SearchFilterParser
import java.time.ZoneId
import org.junit.Assert.assertEquals
import org.junit.Assert.assertNull
import org.junit.Test

```

```

class SearchFilterParserTest {
    private val zoneId = ZoneId.of("UTC")

    @Test
    fun parseBuildsFiltersFromOptionalInputs() {
        val filters = SearchFilterParser.parse(
            description = " Groceries ",
            categoryId = "1",
            minAmount = "10.5",
            maxAmount = "20.5",
            startDate = "2024-09-01",
            endDate = "2024-09-30",
            zoneId = zoneId
        )

        assertEquals("Groceries", filters.description)
        assertEquals(1L, filters.categoryId)
        assertEquals(1_050L, filters.minAmountCents)
        assertEquals(2_050L, filters.maxAmountCents)
        assertEquals(1_725_148_800_000L, filters.startDateEpochMillis)
        assertEquals(1_727_740_799_999L, filters.endDateEpochMillis)
    }

    @Test(expected = java.time.format.DateTimeParseException::class)
    fun parseRejectsInvalidDates() {
        SearchFilterParser.parse(
            description = "",
            categoryId = "",
            minAmount = "",
            maxAmount = "",
            startDate = "09/01/2024",
            endDate = "",
            zoneId = zoneId
        )
    }

    @Test(expected = IllegalArgumentException::class)
    fun parseRejectsInvalidCategoryFilter() {
        SearchFilterParser.parse(
            description = "",
            categoryId = "abc",
            minAmount = "",
            maxAmount = "",
            startDate = "",
            endDate = "",
            zoneId = zoneId
        )
    }

    @Test(expected = IllegalArgumentException::class)
    fun parseRejectsInvalidMinimumAmount() {
        SearchFilterParser.parse(
            description = "",
            categoryId = "",
            minAmount = "1.2.3",
            maxAmount = "",

```

```

        startDate = "",
        endDate = "",
        zoneId = zoneId
    )
}

@Test(expected = IllegalArgumentException::class)
fun parseRejectsInvalidMaximumAmount() {
    SearchFilterParser.parse(
        description = "",
        categoryId = "",
        minAmount = "",
        maxAmount = "4.5.6",
        startDate = "",
        endDate = "",
        zoneId = zoneId
    )
}

@Test(expected = IllegalArgumentException::class)
fun parseRejectsInvertedDateRange() {
    SearchFilterParser.parse(
        description = "",
        categoryId = "",
        minAmount = "",
        maxAmount = "",
        startDate = "2024-09-30",
        endDate = "2024-09-01",
        zoneId = zoneId
    )
}

@Test
fun parseLeavesBlankInputsUnset() {
    val filters = SearchFilterParser.parse(
        description = " ",
        categoryId = "",
        minAmount = "",
        maxAmount = "",
        startDate = "",
        endDate = "",
        zoneId = zoneId
    )

    assertNull(filters.description)
    assertNull(filters.categoryId)
    assertNull(filters.minAmountCents)
    assertNull(filters.maxAmountCents)
    assertNull(filters.startDateEpochMillis)
    assertNull(filters.endDateEpochMillis)
}
}

```

SessionManagerTest.kt

5-minute session timeout (2 tests).

```
package com.denariidolor

import com.denariidolor.util.Constants
import com.denariidolor.util.SessionManager
import org.junit.Assert.assertFalse
import org.junit.Assert.assertTrue
import org.junit.Test

class SessionManagerTest {
    @Test
    fun invalidateMarksSessionAsTimedOut() {
        val sessionManager = SessionManager()
        sessionManager.markAuthenticated(now = 1_000L)

        assertFalse(sessionManager.isSessionTimedOut(now = 1_000L +
            Constants.SESSION_TIMEOUT_MILLIS - 1))

        sessionManager.invalidate()

        assertTrue(sessionManager.isSessionTimedOut(now = 1_000L))
    }

    @Test
    fun touchDoesNotAuthenticateByItself() {
        val sessionManager = SessionManager()

        sessionManager.touch(now = 1_000L)

        assertTrue(sessionManager.isSessionTimedOut(now = 1_000L))
    }
}
```

TransactionCrudUseCaseTest.kt

Add/update/delete transaction use cases (5 tests).

```
package com.denariidolor

import com.denariidolor.domain.model.Expense
import com.denariidolor.domain.model.Income
import com.denariidolor.domain.model.TransactionType
import com.denariidolor.domain.usecase.AddTransactionUseCase
import com.denariidolor.domain.usecase.DeleteTransactionUseCase
import com.denariidolor.domain.usecase.UpdateTransactionUseCase
import com.denariidolor.domain.usecase.ValidateTransactionUseCase
import com.denariidolor.testutil.FakeAccountRepository
import com.denariidolor.testutil.FakeBudgetRepository
import com.denariidolor.testutil.FakeCategoryRepository
import com.denariidolor.testutil.FakeTransactionRepository
import com.denariidolor.testutil.TestData
```

```

import kotlinx.coroutines.runBlocking
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

class TransactionCrudUseCaseTest {
    private val transactions = FakeTransactionRepository(listOf(TestData.expense(id = 1,
amountCents = 1_000)))
    private val validate = ValidateTransactionUseCase(
        FakeCategoryRepository(TestData.categories),
        FakeAccountRepository(TestData.accounts),
        FakeBudgetRepository(),
        transactions
    )
    private val add = AddTransactionUseCase(validate, transactions)
    private val update = UpdateTransactionUseCase(validate, transactions)
    private val delete = DeleteTransactionUseCase(transactions)

    private fun expense(id: Long, amountCents: Long) = Expense(
        id = id,
        description = "Coffee",
        amountCents = amountCents,
        categoryId = 4,
        accountId = 1,
        dateEpochMillis = System.currentTimeMillis()
    )

    @Test
    fun addIgnoresCallerSuppliedId() = runBlocking<Unit> {
        val id = add(expense(id = 1, amountCents = 500)).getOrNull()

        assertEquals(2L, id)
        assertEquals(2, transactions.items.size)
    }

    @Test
    fun updateReplacesExistingTransaction() = runBlocking<Unit> {
        val result =
            update(Income(id = 1, description = "Refund", amountCents = 3_000, categoryId = 2,
accountId = 1, dateEpochMillis = 1L))

        assertTrue(result.isSuccess)
        assertEquals(TransactionType.INCOME, transactions.items.single().type)
        assertEquals(3_000L, transactions.items.single().amountCents)
    }

    @Test
    fun updateRejectsMissingIdAndUnknownTransaction() = runBlocking<Unit> {
        assertEquals("Transaction ID is required", update(expense(id = 0, amountCents =
500)).exceptionOrNull()?.message)
        assertTrue(update(expense(id = 42, amountCents = 500)).exceptionOrNull() is
NoSuchElementException)
    }

    @Test
    fun updateRunsValidation() = runBlocking<Unit> {

```

```

        assertEquals("Invalid amount", update(expense(id = 1, amountCents =
-100)).exceptionOrNull()?.message)
        assertEquals(1_000L, transactions.items.single().amountCents)
    }

    @Test
    fun deleteRemovesTransactionAndReportsMissing() = runBlocking<Unit> {
        assertTrue(delete(1).isSuccess)
        assertTrue(transactions.items.isEmpty())
        assertTrue(delete(1).exceptionOrNull() is NoSuchElementException)
        assertTrue(delete(0).isFailure)
    }
}

```

TransactionEntityMappingsTest.kt

Entity ↔ domain model mapping (2 tests).

```

package com.denariidolor

import com.denariidolor.data.local.db.entity.TransactionEntity
import com.denariidolor.domain.model.TransactionType
import com.denariidolor.domain.model.Transfer
import com.denariidolor.domain.model.toDomainTransaction
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

class TransactionEntityMappingsTest {
    @Test
    fun transferEntityMapsToTransferDomainModel() {
        val transaction = TransactionEntity(
            id = 7,
            type = TransactionType.TRANSFER,
            description = "Move to savings",
            amountCents = 10_000L,
            categoryId = 3,
            accountId = 1,
            transferAccountId = 2,
            dateEpochMillis = 1234L
        ).toDomainTransaction() as Transfer

        assertEquals(2L, transaction.transferAccountId)
        assertEquals(0L, transaction.balanceImpact())
    }

    @Test
    fun transferEntityWithoutDestinationFailsFast() {
        val result = runCatching {
            TransactionEntity(
                id = 7,
                type = TransactionType.TRANSFER,
                description = "Move to savings",
                amountCents = 10_000L,
                categoryId = 3,

```

```

        accountId = 1,
        transferAccountId = null,
        dateEpochMillis = 1234L
    ).toDomainTransaction()
}

assertTrue(result.isFailure)
}
}

```

TransactionModelTest.kt

Transaction subclasses and TransactionType (4 tests).

```

package com.denariidolor

import com.denariidolor.domain.model.Expense
import com.denariidolor.domain.model.Income
import com.denariidolor.domain.model.TransactionType
import com.denariidolor.domain.model.Transfer
import org.junit.Assert.assertEquals
import org.junit.Test

class TransactionModelTest {
    private val expense = Expense(description = "Groceries", amountCents = 4_500, categoryId = 1,
        accountId = 1, dateEpochMillis = 1L)
    private val income = Income(description = "Salary", amountCents = 125_000, categoryId = 2,
        accountId = 1, dateEpochMillis = 1L)
    private val transfer =
        Transfer(description = "Move", amountCents = 8_000, categoryId = 3, accountId = 1,
            transferAccountId = 2, dateEpochMillis = 1L)

    @Test
    fun transactionSubclassesReportExpectedBalanceImpact() {
        assertEquals(-4_500L, expense.balanceImpact())
        assertEquals(125_000L, income.balanceImpact())
        assertEquals(0L, transfer.balanceImpact())
    }

    @Test
    fun transactionSubclassesReportPerAccountImpacts() {
        assertEquals(mapOf(1L to -4_500L), expense.accountImpacts())
        assertEquals(mapOf(1L to 125_000L), income.accountImpacts())
        assertEquals(mapOf(1L to -8_000L, 2L to 8_000L), transfer.accountImpacts())
    }

    @Test
    fun subclassesExposeTheirType() {
        assertEquals(TransactionType.EXPENSE, expense.type)
        assertEquals(TransactionType.INCOME, income.type)
        assertEquals(TransactionType.TRANSFER, transfer.type)
    }

    @Test
    fun typeParsingIsCaseInsensitiveAndStrict() {

```

```

    assertEquals(TransactionType.INCOME, TransactionType.parse(" income "))
    assertEquals(true, runCatching { TransactionType.parse("REFUND") }.isFailure)
  }
}

```

data/local/db/DatabaseKeysTest.kt

SQLCipher key format and handling (5 tests).

```

package com.denariidolor.data.local.db

import com.denariidolor.data.local.db.security.DatabaseKeys
import java.security.SecureRandom
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNotEquals
import org.junit.Assert.assertTrue
import org.junit.Rule
import org.junit.Test
import org.junit.rules.TemporaryFolder

class DatabaseKeysTest {
    @get:Rule
    val temp = TemporaryFolder()

    @Test
    fun generatedKeysAre256BitHexAndUnique() {
        val random = SecureRandom()
        val first = DatabaseKeys.generateHexKey(random)
        val second = DatabaseKeys.generateHexKey(random)

        assertEquals(64, first.length)
        assertTrue(DatabaseKeys.isValidHexKey(first))
        assertNotEquals(first, second)
    }

    @Test
    fun invalidStoredKeysAreRejected() {
        assertFalse(DatabaseKeys.isValidHexKey(""))
        assertFalse(DatabaseKeys.isValidHexKey("zz".repeat(32)))
        assertFalse(DatabaseKeys.isValidHexKey("ab".repeat(31)))
    }

    @Test
    fun passphraseUsesSqlCipherRawKeySyntax() {
        val hex = "ab".repeat(32)

        assertEquals("x'$hex'", String(DatabaseKeys.toRawKeyPassphrase(hex), Charsets.US_ASCII))
    }

    @Test
    fun detectsPlaintextSqliteHeader() {
        val plaintext = temp.newFile("plain.db").apply { writeBytes("SQLite format
3\u0000".toByteArray() + ByteArray(100)) }
    }
}

```

```

        val encrypted = temp.newFile("enc.db").apply { writeBytes(ByteArray(116) { (it *
7).toByte() }) }
        val tiny = temp.newFile("tiny.db").apply { writeBytes(byteArrayOf(1, 2, 3)) }

        assertTrue(DatabaseKeys.isPlaintextDatabase(plaintext))
        assertFalse(DatabaseKeys.isPlaintextDatabase(encrypted))
        assertFalse(DatabaseKeys.isPlaintextDatabase(tiny))
    }

    @Test
    fun discardsPlaintextOrOrphanedDatabasesOnly() {
        val plaintext = temp.newFile("plain.db").apply { writeBytes("SQLite format
3\u0000".toByteArray() + ByteArray(100)) }
        val encrypted = temp.newFile("enc.db").apply { writeBytes(ByteArray(116) { 1 }) }
        val missing = temp.root.resolve("missing.db")

        assertTrue(DatabaseKeys.shouldDiscard(plaintext, keyNewlyCreated = false))
        assertTrue(DatabaseKeys.shouldDiscard(encrypted, keyNewlyCreated = true))
        assertFalse(DatabaseKeys.shouldDiscard(encrypted, keyNewlyCreated = false))
        assertFalse(DatabaseKeys.shouldDiscard(missing, keyNewlyCreated = true))
    }
}

```

data/local/preferences/SecurityProfileServiceTest.kt

PIN hashing, lockout escalation, recovery, wipe (17 tests).

```

package com.denariidolor.data.local.preferences

import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertNull
import org.junit.Assert.assertTrue
import org.junit.Test

class SecurityProfileServiceTest {

    @Test
    fun setupProfileSucceedsOnceAndConfiguresSignIn() {
        val service = SecurityProfileService(InMemorySecurityProfileStore())

        val result = service.setupProfile(
            pin = "1234",
            pinConfirmation = "1234",
            securityQuestion = "Favorite color?",
            securityAnswer = "blue"
        )

        assertEquals(SetupProfileResult.SUCCESS, result)
        assertTrue(service.isProfileConfigured())
        assertEquals(ProfileMode.SIGN_IN, service.getProfileState().mode)
    }

    @Test

```

```

fun repeatedSetupIsRejected() {
    val service = SecurityProfileService(InMemorySecurityProfileStore())
    service.setupProfile("1234", "1234", "Question?", "answer")

    val result = service.setupProfile("1234", "1234", "Question?", "answer")

    assertEquals(SetupProfileResult.ALREADY_CONFIGURED, result)
}

```

@Test

```

fun pinVerificationSucceedsAndFails() {
    val service = SecurityProfileService(InMemorySecurityProfileStore())
    service.setupProfile("1234", "1234", "Question?", "answer")

    assertTrue(service.verifyPin("1234"))
    assertFalse(service.verifyPin("5555"))
}

```

@Test

```

fun recoveryRejectsIncorrectAnswerWithoutChangingPin() {
    val service = SecurityProfileService(InMemorySecurityProfileStore())
    service.setupProfile("1234", "1234", "Question?", "answer")

    val result = service.recoverPin(
        securityAnswer = "wrong",
        newPin = "5678",
        pinConfirmation = "5678"
    )

    assertEquals(RecoverPinResult.INVALID_SECURITY_ANSWER, result)
    assertTrue(service.verifyPin("1234"))
    assertFalse(service.verifyPin("5678"))
}

```

@Test

```

fun recoveryWithCorrectAnswerReplacesPin() {
    val service = SecurityProfileService(InMemorySecurityProfileStore())
    service.setupProfile("1234", "1234", "Question?", "answer")

    val result = service.recoverPin(
        securityAnswer = "answer",
        newPin = "5678",
        pinConfirmation = "5678"
    )

    assertEquals(RecoverPinResult.SUCCESS, result)
    assertFalse(service.verifyPin("1234"))
    assertTrue(service.verifyPin("5678"))
}

```

@Test

```

fun wipeClearsConfiguredState() {
    val service = SecurityProfileService(InMemorySecurityProfileStore())
    service.setupProfile("1234", "1234", "Question?", "answer")

    service.wipeAll()
}

```

```

    assertFalse(service.isProfileConfigured())
    assertFalse(service.verifyPin("1234"))
    assertEquals(ProfileMode.FIRST_TIME_SETUP, service.getProfileState().mode)
}

@Test
fun cancelingWipeKeepsExistingCredentialsUntouched() {
    val service = SecurityProfileService(InMemorySecurityProfileStore())
    service.setupProfile("1234", "1234", "Question?", "answer")

    // Simulates user canceling the destructive dialog by not invoking wipeAll().
    assertTrue(service.isProfileConfigured())
    assertTrue(service.verifyPin("1234"))
}

@Test
fun legacyPinRequiresMatchingPinToMigrateAndClearsLegacyAfterSuccess() {
    val store = InMemorySecurityProfileStore().apply {
        edit { putString(SecurityProfileService.KEY_LEGACY_PIN, "2468") }
    }
    val service = SecurityProfileService(store)

    val mismatch = service.setupProfile(
        pin = "1234",
        pinConfirmation = "1234",
        securityQuestion = "Question?",
        securityAnswer = "answer"
    )
    assertEquals(SetupProfileResult.LEGACY_PIN_MISMATCH, mismatch)
    assertNotNull(store.getString(SecurityProfileService.KEY_LEGACY_PIN))

    val success = service.setupProfile(
        pin = "2468",
        pinConfirmation = "2468",
        securityQuestion = "Question?",
        securityAnswer = "answer"
    )

    assertEquals(SetupProfileResult.SUCCESS, success)
    assertNull(store.getString(SecurityProfileService.KEY_LEGACY_PIN))
    assertTrue(service.verifyPin("2468"))
}

private var now = 1_000_000L

private fun lockableService(): SecurityProfileService =
    SecurityProfileService(InMemorySecurityProfileStore(), clock = { now }).apply {
        setupProfile("1234", "1234", "Question?", "answer")
    }

@Test
fun lockoutDurationEscalatesAndCaps() {
    assertEquals(0L, SecurityProfileService.lockoutDurationMillis(4))
    assertEquals(30_000L, SecurityProfileService.lockoutDurationMillis(5))
    assertEquals(60_000L, SecurityProfileService.lockoutDurationMillis(6))
    assertEquals(480_000L, SecurityProfileService.lockoutDurationMillis(9))
}

```

```

    assertEquals(900_000L, SecurityProfileService.lockoutDurationMillis(10))
    assertEquals(900_000L, SecurityProfileService.lockoutDurationMillis(1_000))
}

@Test
fun fifthWrongPinLocksOutAndBlocksCorrectPin() {
    val service = lockableService()

    repeat(4) { attempt ->
        assertEquals(PinAttemptResult.Invalid(attemptsBeforeLockout = 4 - attempt),
            service.attemptPin("0000"))
    }
    assertEquals(PinAttemptResult.LockedOut(30_000L), service.attemptPin("0000"))

    now += 10_000L
    assertEquals(PinAttemptResult.LockedOut(20_000L), service.attemptPin("1234"))
}

@Test
fun lockoutExpiresAndNextFailureDoubles() {
    val service = lockableService()
    repeat(5) { service.attemptPin("0000") }

    now += 30_000L
    assertEquals(PinAttemptResult.LockedOut(60_000L), service.attemptPin("0000"))
}

@Test
fun successResetsFailureCounter() {
    val service = lockableService()
    repeat(4) { service.attemptPin("0000") }

    assertEquals(PinAttemptResult.Success, service.attemptPin("1234"))
    assertEquals(PinAttemptResult.Invalid(attemptsBeforeLockout = 4),
        service.attemptPin("0000"))
}

@Test
fun biometricSuccessResetsLockout() {
    val service = lockableService()
    repeat(5) { service.attemptPin("0000") }

    service.recordSuccessfulAuthentication()

    assertEquals(0L, service.lockoutRemainingMillis())
    assertEquals(PinAttemptResult.Success, service.attemptPin("1234"))
}

@Test
fun clockRollbackDoesNotShortenLockout() {
    val service = lockableService()
    repeat(5) { service.attemptPin("0000") }

    now -= 3_600_000L

    assertEquals(30_000L, service.lockoutRemainingMillis())
}

```

```

@Test
fun wrongSecurityAnswersShareTheLockout() {
    val service = lockableService()
    repeat(4) { assertEquals(RecoverPinResult.INVALID_SECURITY_ANSWER,
service.recoverPin("wrong", "5678", "5678")) }

    assertEquals(RecoverPinResult.LOCKED_OUT, service.recoverPin("wrong", "5678", "5678"))
    assertEquals(RecoverPinResult.LOCKED_OUT, service.recoverPin("answer", "5678", "5678"))
    assertTrue(service.attemptPin("1234") is PinAttemptResult.LockedOut)
}

@Test
fun successfulRecoveryClearsLockout() {
    val service = lockableService()
    repeat(4) { service.attemptPin("0000") }

    assertEquals(RecoverPinResult.SUCCESS, service.recoverPin("answer", "5678", "5678"))
    assertEquals(PinAttemptResult.Invalid(attemptsBeforeLockout = 4),
service.attemptPin("0000"))
}

@Test
fun wipeClearsLockout() {
    val service = lockableService()
    repeat(5) { service.attemptPin("0000") }

    service.wipeAll()

    assertEquals(0L, service.lockoutRemainingMillis())
}

private class InMemorySecurityProfileStore : SecurityProfileStore {
    private val stringValues = mutableMapOf<String, String>()
    private val intValue = mutableMapOf<String, Int>()
    private val longValues = mutableMapOf<String, Long>()
    private val booleanValues = mutableMapOf<String, Boolean>()

    override fun getString(key: String): String? = stringValues[key]

    override fun getInt(key: String, defaultValue: Int): Int = intValue[key] ?: defaultValue

    override fun getLong(key: String, defaultValue: Long): Long = longValues[key] ?:
defaultValue

    override fun getBoolean(key: String, defaultValue: Boolean): Boolean = booleanValues[key]
?: defaultValue

    override fun edit(block: SecurityProfileStoreEditor.() -> Unit) {
        val editor = object : SecurityProfileStoreEditor {
            override fun putString(key: String, value: String) {
                stringValues[key] = value
            }

            override fun putInt(key: String, value: Int) {
                intValue[key] = value
            }
        }
        block(editor)
    }
}

```

```

        override fun putLong(key: String, value: Long) {
            longValues[key] = value
        }

        override fun putBoolean(key: String, value: Boolean) {
            booleanValues[key] = value
        }

        override fun remove(key: String) {
            stringValues.remove(key)
            intValues.remove(key)
            longValues.remove(key)
            booleanValues.remove(key)
        }

        override fun clear() {
            stringValues.clear()
            intValues.clear()
            longValues.clear()
            booleanValues.clear()
        }
    }
}
block(editor)
}
}
}

```

domain/report/ReportFormattingTest.kt

Report labels, file names, CSV layout and formula-injection guard (4 tests).

```

package com.denariidolor.domain.report

import com.denariidolor.domain.model.MonthlyReport
import com.denariidolor.domain.model.ReportRow
import com.denariidolor.domain.model.TransactionType
import java.time.YearMonth
import java.time.ZoneId
import java.util.Locale
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

class ReportFormattingTest {
    private val utc = ZoneId.of("UTC")
    private val report = MonthlyReport(
        title = ReportText.TITLE,
        period = YearMonth.of(2026, 9),
        generatedAtEpochMillis = 1_789_992_000_000L,
        totalIncomeCents = 200_000,
        totalExpenseCents = 90_050,
        netCents = 109_950,
        rows = listOf(

```

```

        ReportRow(1, 1_788_220_800_000L, TransactionType.EXPENSE, "Dining", "Lunch, \"team\"",
1_250, "Cash"),
        ReportRow(2, 1_788_220_800_000L, TransactionType.EXPENSE, "Misc", "=HYPERLINK(\"x\")",
100, "Visa")
    )
}

@Test
fun labelsAndFileName() {
    assertEquals("September 2026", ReportText.periodLabel(YearMonth.of(2026, 9), Locale.US))
    assertEquals("spending-report-2026-09.pdf", ReportText.fileName(YearMonth.of(2026, 9),
"pdf"))
}

@Test
fun generatedLabelFormatsInZone() {
    assertEquals("1970-01-01 00:00 UTC", ReportText.generatedLabel(0L, utc))
}

@Test
fun csvContainsMetadataHeaderAndRows() {
    val lines = ReportCsvFormatter.format(report, utc, Locale.US).split("\r\n")

    assertEquals("title,Monthly Spending Report", lines[0])
    assertEquals("period,September 2026", lines[1])
    assertTrue(lines[2].startsWith("generated_at,"))
    assertEquals("total_expense,900.50", lines[4])
    assertEquals("date,type,category,description,amount,payment_method", lines[7])
    assertEquals("2026-09-01,EXPENSE,Dining,\"Lunch, \"\"team\"\"\",12.50,Cash", lines[8])
}

@Test
fun csvNeutralizesFormulaInjection() {
    assertEquals("\"'=HYPERLINK(\"x\")\"", ReportCsvFormatter.escape("=HYPERLINK(\"x\")"))
    assertEquals("\"'@cmd\"", ReportCsvFormatter.escape("@cmd"))
    assertEquals("-12.50", ReportCsvFormatter.escape("-12.50"))
    assertEquals("plain", ReportCsvFormatter.escape("plain"))
}
}

```

presentation/ui/CategoryIconsTest.kt

Category icon catalogue (3 tests).

```

package com.denariidolor.presentation.ui

import com.denariidolor.presentation.ui.common.CategoryIcons
import com.denariidolor.util.Constants
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

class CategoryIconsTest {
    @Test
    fun keysAreUniqueAndIncludeDefault() {

```

```

    val keys = CategoryIcons.options.map { it.key }

    assertEquals(keys.size, keys.toSet().size)
    assertTrue(CategoryIcons.isKnown(CategoryIcons.DEFAULT_KEY))
}

@Test
fun unknownOrNullKeysFallbackToDefault() {
    assertEquals(CategoryIcons.DEFAULT_KEY, CategoryIcons.forKey("nope").key)
    assertEquals(CategoryIcons.DEFAULT_KEY, CategoryIcons.forKey(null).key)
}

@Test
fun seededCategoriesUseKnownIcons() {
    Constants.DEFAULT_CATEGORIES.forEach { assertTrue(it.iconName,
        CategoryIcons.isKnown(it.iconName)) }
}
}

```

presentation/ui/DashboardMappersTest.kt

Dashboard summaries, chart data, budget progress (12 tests).

```

package com.denariidolor.presentation.ui

import com.denariidolor.data.local.db.entity.AccountEntity
import com.denariidolor.data.local.db.entity.BudgetEntity
import com.denariidolor.data.local.db.entity.CategoryEntity
import com.denariidolor.data.local.db.entity.TransactionEntity
import com.denariidolor.domain.model.TransactionType
import com.denariidolor.presentation.ui.common.buildTransactionRows
import com.denariidolor.presentation.ui.dashboard.BudgetProgress
import com.denariidolor.presentation.ui.dashboard.BudgetStatus
import com.denariidolor.presentation.ui.dashboard.budgetProgress
import com.denariidolor.presentation.ui.dashboard.budgetStatus
import com.denariidolor.presentation.ui.dashboard.compactMoney
import com.denariidolor.presentation.ui.dashboard.spendingByCategory
import com.denariidolor.presentation.ui.dashboard.summarize
import com.denariidolor.util.formatMoney
import com.denariidolor.util.formatSignedAmount
import java.time.ZoneId
import org.junit.Assert.assertEquals
import org.junit.Test

class DashboardMappersTest {
    private val utc = ZoneId.of("UTC")
    private val categories = listOf(CategoryEntity(id = 1, name = "Dining"), CategoryEntity(id = 3,
        name = "Transfer"))
    private val accounts =
        listOf(AccountEntity(id = 1, name = "Cash", balanceCents = 0L), AccountEntity(id = 2, name
            = "Savings", balanceCents = 0L))

    private fun txn(id: Long, type: TransactionType, date: Long, transferTo: Long? = null,
        categoryId: Long = 1) = TransactionEntity(
        id = id,

```

```

        type = type,
        description = "T$id",
        amountCents = 1_000,
        categoryId = categoryId,
        accountId = 1,
        transferAccountId = transferTo,
        dateEpochMillis = date
    )

@Test
fun rowsAreNewestFirstAndLimited() {
    val rows = buildTransactionRows(
        listOf(txn(1, TransactionType.EXPENSE, 1_000), txn(2, TransactionType.EXPENSE, 3_000),
        txn(3, TransactionType.INCOME, 2_000)),
        categories,
        accounts,
        limit = 2,
        zoneId = utc
    )

    assertEquals(listOf(2L, 3L), rows.map { it.id })
}

@Test
fun rowsResolveNamesAndTransferLabel() {
    val row = buildTransactionRows(
        listOf(txn(1, TransactionType.TRANSFER, 0, transferTo = 2, categoryId = 3)),
        categories,
        accounts,
        zoneId = utc
    ).single()

    assertEquals("Transfer", row.categoryName)
    assertEquals("Cash → Savings", row.accountLabel)
    assertEquals("1970-01-01", row.dateText)
    assertEquals("$10.00", row.amountText)
}

@Test
fun missingReferencesFallBackToIds() {
    val row = buildTransactionRows(
        listOf(txn(1, TransactionType.EXPENSE, 0, categoryId = 99)),
        emptyList(),
        emptyList(),
        zoneId = utc
    ).single()

    assertEquals("#99", row.categoryName)
    assertEquals("#1", row.accountLabel)
}

@Test
fun signedAmountsReflectType() {
    assertEquals("-$4.50", formatSignedAmount(TransactionType.EXPENSE, 450))
    assertEquals("+$1200.00", formatSignedAmount(TransactionType.INCOME, 120_000))
    assertEquals("$80.00", formatSignedAmount(TransactionType.TRANSFER, 8_000))
    assertEquals("$0.50", formatMoney(50))
}

```

```

    assertEquals("-$12.05", formatMoney(-1_205))
}

@Test
fun summaryTreatsTransfersAsNetNeutral() {
    val summary =
        summarize(
            listOf(
                txn(1, TransactionType.INCOME, 0),
                txn(2, TransactionType.EXPENSE, 0),
                txn(3, TransactionType.TRANSFER, 0, transferTo = 2)
            )
        )

    assertEquals(1_000L, summary.incomeCents)
    assertEquals(1_000L, summary.expenseCents)
    assertEquals(0L, summary.netCents)
}

@Test
fun rowsCarryCategoryIcon() {
    val cats = listOf(CategoryEntity(id = 1, name = "Dining", iconName = "dining"))
    val row = buildTransactionRows(listOf(txn(1, TransactionType.EXPENSE, 0)), cats, accounts,
    zoneId = utc).single()

    assertEquals("dining", row.categoryIcon)
}

@Test
fun budgetStatusThresholds() {
    assertEquals(BudgetStatus.OK, budgetStatus(7_999, 10_000, 80))
    assertEquals(BudgetStatus.WARNING, budgetStatus(8_000, 10_000, 80))
    assertEquals(BudgetStatus.WARNING, budgetStatus(10_000, 10_000, 80))
    assertEquals(BudgetStatus.OVER, budgetStatus(10_001, 10_000, 80))
}

@Test
fun spendingSortsDescendingAndFolds0Other() {
    val cats = (1L..7L).map { CategoryEntity(id = it, name = "C${it}") }
    val expenses =
        (1L..7L).map { txn(it, TransactionType.EXPENSE, 0, categoryId = it).copy(amountCents =
it * 1_000) } +
        txn(99, TransactionType.INCOME, 0)

    val spending = spendingByCategory(expenses, cats, maxBars = 5)

    assertEquals(listOf(7L, 6L, 5L, 4L, 3L, null), spending.map { it.categoryId })
    assertEquals(3_000L, spending.last().amountCents)
}

@Test
fun spendingWithoutFoldWhenFewCategories() {
    val spending = spendingByCategory(listOf(txn(1, TransactionType.EXPENSE, 0), txn(2,
TransactionType.EXPENSE, 0)), categories)

    assertEquals(1, spending.size)
    assertEquals(2_000L, spending.single().amountCents)
}

```

```

    }

    @Test
    fun budgetProgressSumsMonthExpensesAndSortsByFraction() {
        val budgets = listOf(
            BudgetEntity(id = 1, categoryId = 1, monthlyLimitCents = 10_000),
            BudgetEntity(id = 2, categoryId = 3, monthlyLimitCents = 1_000, warningThresholdPercent
= 50)
        )
        val progress =
            budgetProgress(
                budgets,
                categories,
                listOf(txn(1, TransactionType.EXPENSE, 0), txn(2, TransactionType.EXPENSE, 0),
txn(3, TransactionType.INCOME, 0))
            )

        assertEquals(listOf(1L, 3L), progress.map { it.categoryId })
        assertEquals(2_000L, progress[0].spentCents)
        assertEquals(0L, progress[1].spentCents)
        assertEquals(BudgetStatus.OK, progress[0].status)
    }

    @Test
    fun compactMoneyAxisLabels() {
        assertEquals("$5", compactMoney(5f))
        assertEquals("$12.5", compactMoney(12.5f))
        assertEquals("$1.5K", compactMoney(1_500f))
        assertEquals("$2.0M", compactMoney(2_000_000f))
    }

    @Test
    fun percentUsedUsesIntegerMath() {
        // 0.95f * 100 truncates to 94 in floating point; the displayed value must be 95.
        assertEquals(95, BudgetProgress(1, "Dining", "dining", spentCents = 9_500, limitCents =
10_000, warningPercent = 80).percentUsed)
        assertEquals(0, BudgetProgress(1, "Dining", "dining", spentCents = 500, limitCents = 0,
warningPercent = 80).percentUsed)
    }
}

```

presentation/ui/FormMappingTest.kt

Form input ↔ entity mapping (4 tests).

```

package com.denariidolor.presentation.ui

import com.denariidolor.data.local.db.entity.BudgetEntity
import com.denariidolor.data.local.db.entity.CategoryEntity
import com.denariidolor.data.local.db.entity.TransactionEntity
import com.denariidolor.domain.model.TransactionType
import com.denariidolor.presentation.ui.account.AccountViewModel
import com.denariidolor.presentation.ui.budget.buildBudgetRows
import com.denariidolor.presentation.ui.transaction.TransactionFormInput
import java.time.ZoneId

```

```

import org.junit.Assert.assertEquals
import org.junit.Assert.assertNull
import org.junit.Test

class FormMappingTest {
    private val entity = TransactionEntity(
        id = 5,
        type = TransactionType.TRANSFER,
        description = "Save",
        amountCents = 25_000,
        categoryId = 3,
        accountId = 1,
        transferAccountId = 2,
        dateEpochMillis = 1_725_192_000_000L
    )

    @Test
    fun formInputFromEntityFormatsFields() {
        val input = TransactionFormInput.from(entity, ZoneId.of("UTC"))

        assertEquals("250", input.amount)
        assertEquals("2", input.transferAccountId)
        assertEquals("2024-09-01", input.dateText)
    }

    @Test
    fun formInputKeepsOriginalTimestampOnlyWhenDateUnchanged() {
        val input = TransactionFormInput.from(entity, ZoneId.of("UTC"))

        assertEquals(entity.dateEpochMillis, input.resolveDateEpochMillis("2024-09-01") {
            error("should not parse") })
        assertEquals(42L, input.resolveDateEpochMillis("2024-09-02") { 42L })
    }

    @Test
    fun budgetRowsPairCategoriesWithBudgets() {
        val rows = buildBudgetRows(
            listOf(CategoryEntity(id = 1, name = "Food"), CategoryEntity(id = 2, name = "Fun")),
            listOf(BudgetEntity(id = 9, categoryId = 2, monthlyLimitCents = 5_000))
        )

        assertNull(rows[0].budget)
        assertEquals(9L, rows[1].budget?.id)
    }

    @Test
    fun openingBalanceParsing() {
        assertEquals(0L, AccountViewModel.parseOpeningBalance(" "))
        assertEquals(-2_550L, AccountViewModel.parseOpeningBalance("-25.5"))
        assertNull(AccountViewModel.parseOpeningBalance("abc"))
        assertNull(AccountViewModel.parseOpeningBalance("1.005"))
    }
}

```

presentation/ui/TransactionTypeDefaultsTest.kt

Form defaults per transaction type (2 tests).

```
package com.denariidolor.presentation.ui

import com.denariidolor.util.Constants
import org.junit.Assert.assertEquals
import org.junit.Test

class TransactionTypeDefaultsTest {
    @Test
    fun `auto category is replaced when prior value was auto generated`() {
        val defaults = applyTransactionTypeDefaults(
            selectedType = "INCOME",
            currentCategoryId = Constants.DEFAULT_EXPENSE_CATEGORY_ID.toString(),
            lastAutoCategoryId = Constants.DEFAULT_EXPENSE_CATEGORY_ID.toString(),
            accountId = ""
        )

        assertEquals(Constants.DEFAULT_INCOME_CATEGORY_ID.toString(), defaults.categoryId)
        assertEquals(Constants.DEFAULT_CASH_ACCOUNT_ID.toString(), defaults.accountId)
        assertEquals("", defaults.transferAccountId)
    }

    @Test
    fun `manual category is preserved while transfer defaults are applied`() {
        val defaults = applyTransactionTypeDefaults(
            selectedType = "TRANSFER",
            currentCategoryId = "99",
            lastAutoCategoryId = Constants.DEFAULT_EXPENSE_CATEGORY_ID.toString(),
            accountId = "7"
        )

        assertEquals("99", defaults.categoryId)
        assertEquals("7", defaults.accountId)
        assertEquals(Constants.DEFAULT_SAVINGS_ACCOUNT_ID.toString(), defaults.transferAccountId)
        assertEquals(Constants.DEFAULT_TRANSFER_CATEGORY_ID.toString(),
            defaults.lastAutoCategoryId)
    }
}
```

presentation/viewmodel/DashboardViewModelTest.kt

Dashboard ViewModel state (1 tests).

```
package com.denariidolor.presentation.viewmodel

import com.denariidolor.data.local.db.entity.BudgetEntity
import com.denariidolor.domain.usecase.DeleteTransactionUseCase
import com.denariidolor.presentation.ui.dashboard.BudgetStatus
import com.denariidolor.presentation.ui.dashboard.DashboardViewModel
import com.denariidolor.testutil.FakeAccountRepository
import com.denariidolor.testutil.FakeBudgetRepository
```

```

import com.denariidolor.testutil.FakeCategoryRepository
import com.denariidolor.testutil.FakeTransactionRepository
import com.denariidolor.testutil.MainDispatcherRule
import com.denariidolor.testutil.TestData
import com.denariidolor.util.DateUtils
import java.time.Clock
import java.time.Instant
import java.time.YearMonth
import java.time.ZoneOffset
import kotlin.coroutines.flow.first
import kotlin.coroutines.launch
import kotlin.coroutines.test.runTest
import org.junit.Assert.assertEquals
import org.junit.Rule
import org.junit.Test

class DashboardViewModelTest {
    @get:Rule
    val mainDispatcherRule = MainDispatcherRule()

    private val clock = Clock.fixed(Instant.parse("2026-09-20T12:00:00Z"), ZoneOffset.UTC)
    private val september = DateUtils.monthRangeEpochMillis(2026, 9, ZoneOffset.UTC).first

    @Test
    fun summarizesCurrentMonthOnlyAndFlagsBudgets() = runTest {
        val transactions = FakeTransactionRepository(
            listOf(
                TestData.expense(id = 1, amountCents = 9_000).copy(dateEpochMillis = september +
1),
                TestData.expense(id = 2, amountCents = 50_000).copy(dateEpochMillis = september -
1)
            )
        )
        val viewModel = DashboardViewModel(
            transactions,
            FakeCategoryRepository(TestData.categories),
            FakeAccountRepository(TestData.accounts),
            FakeBudgetRepository(listOf(BudgetEntity(id = 1, categoryId = 4, monthlyLimitCents =
10_000))),
            DeleteTransactionUseCase(transactions),
            clock
        )
        val collector = launch { viewModel.uiState.collect {} }

        val state = viewModel.uiState.first { it.period != null }

        assertEquals(YearMonth.of(2026, 9), state.period)
        assertEquals(9_000L, state.summary.expenseCents)
        assertEquals(-9_000L, state.summary.netCents)
        assertEquals(BudgetStatus.WARNING, state.budgets.single().status)
        assertEquals(1, state.budgetAlerts)
        assertEquals(2, state.recent.size)
        collector.cancel()
    }
}

```

presentation/viewmodel/TransactionViewModelTest.kt

Transaction ViewModel save/validation events (4 tests).

```
package com.denariidolor.presentation.viewmodel

import androidx.lifecycle.SavedStateHandle
import com.denariidolor.domain.usecase.AddTransactionUseCase
import com.denariidolor.domain.usecase.UpdateTransactionUseCase
import com.denariidolor.domain.usecase.ValidateTransactionUseCase
import com.denariidolor.presentation.ui.transaction.TransactionEvent
import com.denariidolor.presentation.ui.transaction.TransactionFormState
import com.denariidolor.presentation.ui.transaction.TransactionViewModel
import com.denariidolor.testutil.FakeAccountRepository
import com.denariidolor.testutil.FakeBudgetRepository
import com.denariidolor.testutil.FakeCategoryRepository
import com.denariidolor.testutil.FakeTransactionRepository
import com.denariidolor.testutil.MainDispatcherRule
import com.denariidolor.testutil.TestData
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.test.runTest
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Rule
import org.junit.Test

class TransactionViewModelTest {
    @get:Rule
    val mainDispatcherRule = MainDispatcherRule()

    private val transactions = FakeTransactionRepository(listOf(TestData.expense(id = 7,
        amountCents = 450)))

    private fun viewModel(transactionId: Long? = null): TransactionViewModel {
        val categories = FakeCategoryRepository(TestData.categories)
        val accounts = FakeAccountRepository(TestData.accounts)
        val validate = ValidateTransactionUseCase(categories, accounts, FakeBudgetRepository(),
            transactions)
        return TransactionViewModel(
            AddTransactionUseCase(validate, transactions),
            UpdateTransactionUseCase(validate, transactions),
            transactions,
            categories,
            accounts,
            SavedStateHandle(transactionId?.let { mapOf(TransactionViewModel.ARG_TRANSACTION_ID to
                it) } ?: emptyMap())
        )
    }

    @Test
    fun editModeLoadsExistingTransaction() = runTest {
        val vm = viewModel(transactionId = 7)

        val state = vm.formState.first { it !is TransactionFormState.Loading } as
            TransactionFormState.Ready
    }
}
```

```

        assertTrue(vm.isEditMode)
        assertEquals("4.5", state.initial?.amount)
        assertEquals("EXPENSE", state.initial?.type)
    }

    @Test
    fun unknownIdReportsNotFound() = runTest {
        assertEquals(TransactionFormState.NotFound, viewModel(transactionId = 99).formState.first {
            it !is TransactionFormState.Loading })
    }

    @Test
    fun addSavesInCentsAndEmitsSaved() = runTest {
        val vm = viewModel()

        vm.saveTransaction("expense", "Lunch", 1_299, categoryId = 4, accountId = 1,
            transferAccountId = null, dateEpochMillis = 1L)

        assertEquals(TransactionEvent.Saved, vm.events.first())
        assertEquals(1_299L, transactions.items.last().amountCents)
    }

    @Test
    fun invalidTransferEmitsFailure() = runTest {
        val vm = viewModel()

        vm.saveTransaction("TRANSFER", "Move", 100, categoryId = 3, accountId = 1,
            transferAccountId = null, dateEpochMillis = 1L)

        assertEquals(TransactionEvent.Failed("Transfer destination is required"),
            vm.events.first())
    }
}

```

testutil/MainDispatcherRule.kt

JUnit rule that swaps Dispatchers.Main for ViewModel tests (0 tests).

```

package com.denariidolor.testutil

import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.ExperimentalCoroutinesApi
import kotlinx.coroutines.test.TestDispatcher
import kotlinx.coroutines.test.UnconfinedTestDispatcher
import kotlinx.coroutines.test.resetMain
import kotlinx.coroutines.test.setMain
import org.junit.rules.TestWatcher
import org.junit.runner.Description

@OptIn(ExperimentalCoroutinesApi::class)
class MainDispatcherRule(val dispatcher: TestDispatcher = UnconfinedTestDispatcher()) :
    TestWatcher() {
    override fun starting(description: Description) = Dispatchers.setMain(dispatcher)
}

```

```
override fun finished(description: Description) = Dispatchers.resetMain()  
}
```